

实验报告：微信小程序开发

实验项目基本信息

- 实验编号：d20301035104、d20301009204
- 学时分配：4学时
- 实验类型：验证型
- 每组人数：6人
- 对应课程目标：课程目标2
- 实验日期：2026年4月
- 姓名：汪明昊
- 学号：2023210624

实验概述

实验目标

本实验旨在开发"智慧校园通知"微信小程序，实现以下核心功能：

- 通知列表展示（分类筛选、已读/未读标记）
- 通知详情查看（富文本内容、附件信息展示）
- 搜索功能（按标题/内容关键词搜索）
- 管理员预览功能（模拟通知发布预览）
- 本地存储（已读状态持久化）

核心步骤

阶段	主要内容	关键技术点
需求分析	定义功能需求与数据模型	页面流程图、数据结构设计
AI辅助编码	使用TRAE生成WXML/WXSS/JS代码	声明式UI、数据绑定、列表渲染
页面开发	列表页、详情页、搜索页、预览页	页面路由、参数传递、条件渲染
本地存储	已读状态持久化	wx.setStorageSync / wx.getStorageSync

阶段	主要内容	关键技术点
测试验证	微信开发者工具预览与功能测试	真机预览、功能验证

✓

成功标准

序号	功能模块	完成状态	备注
1	通知列表展示	✓	20条模拟数据、分类筛选、时间倒序
2	通知详情页	✓	内容展示、附件列表、已读标记
3	分类筛选	✓	全部/公告/教学/活动/后勤/紧急通知
4	搜索功能	✓	关键词搜索标题和内容
5	预览功能	✓	管理员预览界面
6	本地存储	✓	已读状态持久化
7	未读标记	✓	红色圆点未读标识

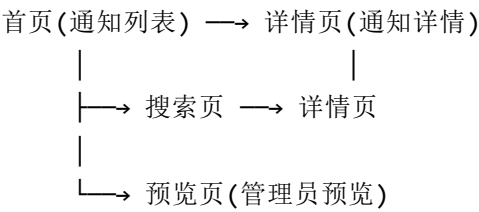
一、需求分析阶段

1.1 功能需求定义

核心功能模块：

模块	功能描述	数据项
通知列表	展示校园通知列表，支持分类筛选	标题、发布部门、时间、分类、已读状态
分类筛选	按通知类型过滤列表	全部、公告、教学、活动、后勤、紧急通知
通知详情	查看通知完整内容和附件	标题、内容、作者、时间、分类、附件列表
搜索功能	按关键词搜索通知	搜索关键词、搜索结果列表
预览功能	管理员发布前预览通知	模拟预览数据、富文本内容
本地存储	已读状态持久化保存	readIds数组

1.2 页面流程图



1.3 数据模型设计

```
// 通知数据结构
{
  id: Number,           // 通知ID
  title: String,        // 通知标题
  content: String,      // 通知内容（支持换行）
  author: String,       // 发布部门
  time: String,         // 发布时间
  category: String,     // 分类标识（公告/教学/活动/后勤/紧急通知）
  categoryText: String, // 分类显示文本
  isRead: Boolean,      // 是否已读
  attachments: [        // 附件列表
    { name: String, url: String }
  ]
}

// 分类数据结构
{
  id: String,           // 分类标识
  name: String          // 分类名称
}

// 本地存储结构
{
  readIds: [Number]     // 已读通知ID数组
}
```

二、项目创建与环境配置

2.1 开发环境

项目	版本
微信开发者工具	最新版
小程序基础库	3.0+
编程语言	JavaScript
界面语言	WXML + WXSS
AI辅助工具	TRAE

2.2 项目结构

— app.js	# 应用入口，初始化本地存储
— app.json	# 应用配置，页面路由定义
— app.wxss	# 全局样式
— project.config.json	# 项目配置文件
— sitemap.json	# 站点地图配置
— data/	
— index.js	# 共享数据模块（通知数据、分类数据、工具函数）
— pages/	
— index/	
— index.wxml	# 首页布局（通知列表）
— index.js	# 首页逻辑（分类筛选、数据加载）
— index.wxss	# 首页样式
— index.json	# 首页配置（下拉刷新）
— detail/	
— detail.wxml	# 详情页布局
— detail.js	# 详情页逻辑（已读标记）
— detail.wxss	# 详情页样式
— detail.json	# 详情页配置
— search/	
— search.wxml	# 搜索页布局
— search.js	# 搜索页逻辑（关键词过滤）
— search.wxss	# 搜索页样式
— search.json	# 搜索页配置
— preview/	
— preview.wxml	# 预览页布局
— preview.js	# 预览页逻辑
— preview.wxss	# 预览页样式
— preview.json	# 预览页配置

2.3 应用配置

```
// app.json - 页面路由与窗口配置
{
  "pages": [
    "pages/index/index",
    "pages/detail/detail",
    "pages/search/search",
    "pages/preview/preview"
  ],
  "window": {
    "backgroundTextStyle": "light",
    "navigationBarBackgroundColor": "#1677FF",
    "navigationBarTitleText": "智慧校园通知",
    "navigationBarTextStyle": "white"
  }
}
```

三、AI辅助编码过程

3.1 AI工具使用记录

步骤	需求描述	AI生成内容	人工完善
1	微信小程序通知列表页，支持分类筛选	基础WXML布局 + JS逻辑	调整样式、添加加载状态
2	通知详情页，接收参数展示内容	详情页布局代码	添加附件列表、完善已读逻辑
3	搜索功能，关键词搜索通知标题和内容	搜索页代码框架	完善数据源、搜索结果展示
4	管理员预览功能	预览页面代码	添加模拟数据、完善UI
5	小程序配置和路由	app.json页面路由	完善窗口配置、下拉刷新配置
6	样式修复	样式冲突问题诊断	清理全局样式冲突、简化布局

3.2 核心代码实现

3.2.1 应用入口（app.js）

```
// app.js
App({
  onLaunch() {
    // 初始化本地存储
    this.initStorage();
  },

  // 初始化存储
  initStorage() {
    if (!wx.getStorageSync('readIds')) {
      wx.setStorageSync('readIds', []);
    }
  },

  // 全局数据
  globalData: {
    userInfo: null
  }
})
```

3.2.2 通知列表页 (pages/index/index.wxml)

```
<!-- pages/index/index.wxml -->
<view class="container">
  <!-- 搜索按钮 -->
  <view class="search-btn-container">
    <button bindtap="goToSearch" class="search-btn">
      <text>🔍 搜索通知</text>
    </button>
  </view>

  <!-- 分类标签 -->
  <view class="category-tabs">
    <view
      wx:for="{{categories}}"
      wx:key="id"
      class="category-tab {{activeCategory === item.id ? 'active' : ''}}"
      bindtap="switchCategory"
      data-id="{{item.id}}"
    >
      <text>{{item.name}}</text>
    </view>
  </view>

  <!-- 通知列表 -->
  <view class="news-list">
    <view wx:if="{{loading}}" class="loading">
      <text>加载中...</text>
    </view>
    <view wx:elif="{{filteredNews.length === 0}}" class="empty">
      <text>暂无通知</text>
    </view>
    <view
      wx:else
      wx:for="{{filteredNews}}"
      wx:key="id"
      class="news-item {{item.isRead ? 'read' : ''}}"
      bindtap="goToDetail"
      data-id="{{item.id}}"
    >
      <view class="news-left">
        <view class="category-tag {{item.category}}">{{item.categoryText}}</view>
        <view class="title">{{item.title}}</view>
        <view class="info">
          <text>{{item.author}}</text>
        </view>
      </view>
    </view>
  </view>
</view>
```



```
        <text>{{item.time}}</text>
    </view>
</view>
<view class="news-right">
    <view wx:if="{{!item.isRead}}" class="unread-dot"></view>
    <view class="arrow">></view>
</view>
</view>
</view>

<!-- 底部版权 -->
<view class="footer">
    <text>© 2024 智慧校园通知系统</text>
</view>
</view>
```

3.2.3 通知列表逻辑 (pages/index/index.js)

```
// pages/index/index.js
Page({
  data: {
    categories: [
      { id: 'all', name: '全部' },
      { id: '公告', name: '公告' },
      { id: '教学', name: '教学' },
      { id: '活动', name: '活动' },
      { id: '后勤', name: '后勤' },
      { id: '紧急通知', name: '紧急' }
    ],
    activeCategory: 'all',
    newsList: [],
    filteredNews: [],
    loading: false
  },

  onLoad() {
    try {
      this.loadNews();
    } catch (error) {
      console.error('页面加载失败:', error);
      this.setData({ loading: false });
      wx.showToast({ title: '加载失败', icon: 'none' });
    }
  },

  onPullDownRefresh() {
    this.loadNews().then(() => {
      wx.stopPullDownRefresh();
    }).catch(() => {
      wx.stopPullDownRefresh();
    });
  },

  // 加载通知数据（20条模拟数据）
  loadNews() {
    this.setData({ loading: true });

    return new Promise((resolve) => {
      setTimeout(() => {
        try {
          const newsList = [
```

```

    { id: 1, title: "关于期末考试安排的通知", content: "...", author: "教务处", time: "2024-01-01" },
    { id: 2, title: "图书馆开放时间调整", content: "...", author: "图书馆", time: "2024-01-02" },
    // ...共计20条模拟数据
    { id: 20, title: "2024年毕业生学位授予仪式通知", content: "...", author: "研究生院", time: "2024-01-20" }
  ];

  // 按时间倒序排列
  newsList.sort((a, b) => new Date(b.time) - new Date(a.time));

  // 读取本地存储的已读状态
  const readIds = wx.getStorageSync('readIds') || [];
  newsList.forEach(news => {
    news.isRead = readIds.includes(news.id);
  });

  this.setData({
    newsList,
    filteredNews: newsList,
    loading: false
  });

  resolve();
} catch (error) {
  console.error('加载数据失败:', error);
  this.setData({ loading: false });
  wx.showToast({ title: '加载失败', icon: 'none' });
  resolve();
}
}, 300);
});
},

// 切换分类
switchCategory(e) {
  const categoryId = e.currentTarget.dataset.id;
  this.setData({ activeCategory: categoryId });

  if (categoryId === 'all') {
    this.setData({ filteredNews: this.data.newsList });
  } else {
    const filteredNews = this.data.newsList.filter(news => news.category === categoryId);
    this.setData({ filteredNews });
  }
},

```

```
// 跳转到详情页
goToDetail(e) {
  const id = e.currentTarget.dataset.id;
  wx.navigateTo({
    url: `/pages/detail/detail?id=${id}`
  });
},

// 跳转到搜索页面
goToSearch() {
  wx.navigateTo({
    url: '/pages/search/search'
  });
}
});
```

3.2.4 通知列表样式 (pages/index/index.wxss)

```
/* pages/index/index.wxss */
.container {
  background-color: #f5f7fa;
  min-height: 100vh;
  padding: 20rpx;
  padding-bottom: 100rpx;
}

/* 搜索按钮 */
.search-btn {
  display: flex;
  align-items: center;
  justify-content: center;
  background-color: #1677FF;
  color: #fff;
  border: none;
  border-radius: 8rpx;
  padding: 24rpx;
  font-size: 32rpx;
  width: 100%;
}

/* 分类标签 - 横向滚动 */
.category-tabs {
  display: flex;
  overflow-x: auto;
  white-space: nowrap;
  margin-bottom: 20rpx;
}

.category-tab {
  flex-shrink: 0;
  padding: 12rpx 28rpx;
  margin-right: 16rpx;
  background-color: #fff;
  border-radius: 32rpx;
  font-size: 30rpx;
  color: #333;
}

.category-tab.active {
  background-color: #1677FF;
  color: #fff;
}
```

```
}

/* 通知列表 */
.news-list {
  background-color: #fff;
  border-radius: 12rpx;
  overflow: hidden;
  box-shadow: 0 2rpx 10rpx rgba(0, 0, 0, 0.05);
  margin-bottom: 40rpx;
}

.news-item {
  display: flex;
  padding: 24rpx;
  border-bottom: 1rpx solid #f5f5f5;
}

.news-item.read {
  opacity: 0.6;
}

/* 分类标签颜色 - 按类型区分 */
.category-tag.公告 { background-color: #E6F7FF; color: #1677FF; }
.category-tag.教学 { background-color: #F6FFED; color: #52C41A; }
.category-tag.活动 { background-color: #FFF7E6; color: #FA8C16; }
.category-tag.后勤 { background-color: #F0F5FF; color: #722ED1; }
.category-tag.紧急通知 { background-color: #FFF1F0; color: #FF4D4F; }

/* 标题 - 最多显示2行，超出省略 */
.title {
  font-size: 32rpx;
  font-weight: 500;
  color: #333;
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

/* 未读红点 */
.unread-dot {
  width: 16rpx;
  height: 16rpx;
  background-color: #FF4D4F;
  border-radius: 50%;
}
```

```
margin-bottom: 16rpx;
}
```

3.2.5 共享数据模块（data/index.js）

```
// data/index.js - 共享数据模块
// 统一管理通知数据，避免在多个页面重复定义

// 通知分类数据
export const categories = [
  { id: 'all', name: '全部' },
  { id: '公告', name: '公告' },
  { id: '教学', name: '教学' },
  { id: '活动', name: '活动' },
  { id: '后勤', name: '后勤' },
  { id: '紧急通知', name: '紧急' }
];

// 通知列表数据（20条模拟数据）
export const newsList = [
  { id: 1, title: "关于期末考试安排的通知", content: "...", author: "教务处", time: "2024-01-01", cat
  // ... 共计20条数据
  { id: 20, title: "2024年毕业生学位授予仪式通知", content: "...", author: "研究生院", time: "2024-03-
];

// 根据ID获取通知详情
export function getNewsById(id) {
  return newsList.find(news => news.id === id);
}

// 根据分类筛选通知
export function filterNewsByCategory(newsList, categoryId) {
  if (categoryId === 'all') {
    return [...newsList];
  }
  return newsList.filter(news => news.category === categoryId);
}

// 根据关键词搜索通知
export function searchNews(keyword) {
  return newsList.filter(news =>
    news.title.includes(keyword) || news.content.includes(keyword)
  );
}
```

设计说明：采用关注点分离原则，将数据层与业务逻辑层分离，通过模块化导出实现代码复用，便于后续维护和扩展。

3.2.6 通知详情页（pages/detail/detail.js）

```
// pages/detail/detail.js
// 引入共享数据模块
const { getNewsById } = require('../../data/index.js');

Page({
  data: {
    news: {}
  },

  onLoad(options) {
    const id = parseInt(options.id);
    this.loadDetail(id);
  },

  // 加载通知详情
  loadDetail(id) {
    // 使用共享数据模块获取通知详情
    const news = getNewsById(id);
    this.setData({ news });

    // 标记为已读
    this.markAsRead(id);
  },

  // 标记为已读
  markAsRead(id) {
    let readIds = wx.getStorageSync('readIds') || [];
    if (!readIds.includes(id)) {
      readIds.push(id);
      wx.setStorageSync('readIds', readIds);
    }
  }
});
```

改进说明：通过引入共享数据模块，消除了数据重复定义，代码更加简洁，维护成本降低。

3.2.6 详情页布局 (pages/detail/detail.wxml)

```
<!-- pages/detail/detail.wxml -->
<view class="container">
  <!-- 通知标题 -->
  <view class="title">{{news.title}}</view>

  <!-- 通知信息 -->
  <view class="info">
    <text>发布部门: {{news.author}}</text>
    <text>发布时间: {{news.time}}</text>
  </view>

  <!-- 分类标签 -->
  <view class="category-tag {{news.category}}">{{news.categoryText}}</view>

  <!-- 通知内容 -->
  <view class="content">{{news.content}}</view>

  <!-- 附件提示 -->
  <view wx:if="{{news.attachments && news.attachments.length > 0}}" class="attachments">
    <text class="attachment-title">附件: </text>
    <view wx:for="{{news.attachments}}" wx:key="index" class="attachment-item">
      <icon type="download" size="20" color="#1677FF"></icon>
      <text>{{item.name}}</text>
    </view>
  </view>
</view>
```

3.2.7 搜索页 (pages/search/search.js)

```
// pages/search/search.js
// 引入共享数据模块
const { searchNews } = require('../../data/index.js');

Page({
  data: {
    keyword: '',
    searchResults: [],
    loading: false
  },

  // 关键词输入变化
  onKeywordChange(e) {
    this.setData({ keyword: e.detail.value });
  },

  // 清除关键词
  clearKeyword() {
    this.setData({ keyword: '' });
  },

  // 搜索操作
  search() {
    const keyword = this.data.keyword.trim();
    if (!keyword) {
      wx.showToast({ title: '请输入搜索关键词', icon: 'none' });
      return;
    }

    this.setData({ loading: true });

    // 模拟搜索延迟
    setTimeout(() => {
      // 使用共享数据模块的搜索函数
      const searchResults = searchNews(keyword);

      // 读取已读状态
      const readIds = wx.getStorageSync('readIds') || [];
      searchResults.forEach(news => {
        news.isRead = readIds.includes(news.id);
      });

      this.setData({ searchResults, loading: false });
    }, 1000);
  }
});
```

```

    }, 500);
  },

  // 跳转到详情页
  goToDetail(e) {
    const id = e.currentTarget.dataset.id;
    wx.navigateTo({ url: `/pages/detail/detail?id=${id}` });
  }
});

```

改进说明：通过引入共享数据模块的searchNews函数，消除了getAllNews()方法的重复定义，代码更加简洁。

3.2.8 预览页（pages/preview/preview.js）

```

// pages/preview/preview.js
Page({
  data: {
    news: {}
  },

  onLoad(options) {
    this.loadPreviewData();
  },

  // 加载预览数据
  loadPreviewData() {
    const previewNews = {
      id: 999,
      title: "关于开展2024年校园科技节的通知（预览）",
      content: "为丰富校园文化生活，激发学生创新精神，学校将于2024年4月15日-20日举办校园科技节，活动内容：",
      author: "科技处",
      time: "2024-03-20",
      category: "活动",
      categoryText: "活动",
      attachments: [
        { name: "科技节活动方案.docx", url: "#" },
        { name: "报名表格.xlsx", url: "#" }
      ]
    };

    this.setData({ news: previewNews });
  }
});

```

四、测试与验证

4.1 测试环境

设备	型号	系统版本	测试方式
微信开发者工具	最新版	Windows	模拟器预览
测试网络	Wi-Fi	-	数据模拟

4.2 功能测试结果

测试项	测试步骤	预期结果	实际结果	状态
通知列表展示	打开小程序首页	显示20条通知列表	正常显示， 含分类标签和未读标记	✓
分类筛选	点击"教学"标签	仅显示教学类通知	正确过滤到8条教学通知	✓
分类切换	从"教学"切回"全部"	恢复显示全部20条	正确恢复	✓
详情跳转	点击任意通知	跳转到详情页	正常跳转， 显示标题和内容	✓
已读标记	查看通知后返回	该通知显示为已读	透明度降低，红点消失	✓
已读持久化	关闭小程序再打开	已读状态保持	状态正确持久化	✓
搜索功能	输入"考试"搜索	显示含"考试"的通知	正确搜索到相关通知	✓
空搜索提示	不输关键词点搜索	提示"请输入搜索关键词"	正常提示	✓
预览功能	进入预览页面	显示模拟通知数据	正常显示预览内容	✓
下拉刷新	下拉首页	重新加载数据	正常刷新	✓
空状态展示	选择无数据的分类	显示"暂无通知"	正常展示空状态	✓
加载状态	首次进入页面	显示"加载中..."	显示加载提示后展示数据	✓

4.3 数据流验证

用户操作流程：

1. 打开小程序 → `app.js` 初始化存储 → 首页 `onLoad` → `loadNews()`
2. `loadNews()` → `setTimeout(300ms)` → `setData({newsList, filteredNews})` → WXML渲染
3. 点击通知 → `goToDetail(id)` → `wx.navigateTo` → 详情页 `onLoad(options)`
4. 详情页 `loadDetail(id)` → `allNews.find()` → `setData({news})` → `markAsRead(id)`
5. `markAsRead` → `wx.getStorageSync('readIds')` → `push(id)` → `wx.setStorageSync('readIds')`
6. 返回首页 → 列表重新读取`readIds` → 已读状态更新

分类筛选流程：

用户点击分类标签 → `switchCategory` → 根据`category`过滤`newsList` → `setData({filteredNews})` → WXML更新渲染

搜索流程：

输入关键词 → `search()` → `getAllNews()` → `filter(keyword)` → `setData({searchResults})` → WXML渲染结果

4.4 真机预览截图

图1：智慧校园通知首页



智慧校园通知



 搜索通知

全部

公告

教学

活动

后勤

紧急

公告

2024年毕业生学位授予仪式通知

研究生院

2024-03-10

紧急

校园网络升级通知

信息中心

2024-03-08

教学

关于调整课程安排的通知

教务处

2024-03-05

活动

2024年暑期社会实践活动通知

学生处

2024-03-01

紧急

校园用水用电安全提醒

后勤处

2024-02-25

公告



功能说明：展示通知列表页面，包含顶部搜索入口、分类标签横向滚动栏（全部/公告/教学/活动/后勤/紧急通知）、20条通知列表（按时间倒序排列），每条通知显示分类标签、标题、发布部门和发布时间。未读通知显示红色圆点标记，已读通知透明度降低。

五、技术与总结

5.1 微信小程序核心机制分析

WXML数据绑定：

微信小程序采用Mustache语法（双大括号 `{{}}`）进行数据绑定，支持：

- 内容绑定： `<text>{{title}}</text>`
- 属性绑定： `class="news-item {{item.isRead ? 'read' : ''}}"`
- 条件渲染： `wx:if`、 `wx:elif`、 `wx:else`
- 列表渲染： `wx:for="{{filteredNews}}" wx:key="id"`
- 事件绑定： `bindtap="goToDetail"`、 `data-id="{{item.id}}"`

页面路由：

使用 `wx.navigateTo()` 进行页面跳转，通过URL参数传递数据：

```
wx.navigateTo({ url: `/pages/detail/detail?id=${id}` });
```

接收参数通过 `onLoad(options)` 中的 `options` 对象获取。

本地存储：

API	功能	特点
<code>wx.setStorageSync</code>	同步写入	立即生效，简单直接

API	功能	特点
wx.getStorageSync	同步读取	快速获取，适合初始化
wx.setStorage	异步写入	不阻塞UI，适合大数据量
wx.getStorage	异步读取	不阻塞UI

5.2 AI辅助开发体验分析

开发环节	AI辅助效果	人工参与程度	体验评价
页面布局生成	优秀	低（微调样式）	★★★★★
JS逻辑编写	良好	中（完善细节）	★★★★
样式设计	良好	中（调整颜色间距）	★★★★
数据处理	良好	中（补充数据项）	★★★★
错误处理	一般	高（补充try-catch）	★★★
Bug修复	优秀	低（按AI建议修复）	★★★★★
文档生成	优秀	低（格式微调）	★★★★★

5.3 开发过程中遇到的问题与解决方案

问题	原因	解决方案
小程序无法显示内容	缺少页面配置文件(index.json)	创建index.json并添加正确配置
样式冲突	全局app.wxss和页面wxss重复定义同名类	清理app.wxss中与页面冲突的样式
图片加载失败	外部placeholder图片在小程序中无法加载	移除外部图片， 改用文字图标和纯CSS组件
通知内容不显示	换行符未正确处理	使用white-space: pre-wrap保留换行
已读状态不更新	返回首页时未重新读取存储	在详情页正确保存readIds

5.4 提示词工程实践

初级提示词（效果不佳）：

帮我写一个小程序

优化后的提示词（高效生成）：

请帮我创建一个微信小程序"智慧校园通知"的通知列表页，具体要求如下：

- 1. 技术栈：微信小程序原生（WXML + WXSS + JS）
- 2. 功能要求：
 - 展示20条模拟通知数据
 - 支持按全部分类筛选（公告/教学/活动/后勤/紧急通知）
 - 点击通知跳转详情页
 - 未读通知显示红色圆点
 - 已读通知透明度降低
 - 支持下拉刷新
- 3. 数据要求：使用模拟数据，从本地存储读取已读状态
- 4. 界面要求：简洁美观，分类标签支持横向滚动
- 5. 输出格式：分别生成index.wxml、index.js、index.wxss三个文件的完整代码

提示词优化心得：

- 1. 明确平台和技术栈（微信小程序原生）
- 2. 列出具体的功能需求（避免模糊描述）
- 3. 提供数据结构参考
- 4. 指定输出格式（分文件输出）
- 5. 补充样式和交互细节要求

六、实验总结

6.1 学习成果

知识点	掌握程度	备注
WXML声明式UI	✅ 熟练	能够独立编写页面布局，掌握条件渲染和列表渲染
WXSS样式编写	✅ 熟练	掌握rpx单位、Flex布局、多类名样式
页面路由与导航	✅ 熟练	掌握wx.navigateTo及参数传递
本地存储API	✅ 熟练	掌握同步/异步存储读写
点击事件处理	✅ 熟练	掌握bindtap及事件传参

知识点	掌握程度	备注
数据绑定与状态管理	✅ 熟练	掌握setData更新视图
AI辅助开发	✅ 掌握	能够使用TRAE进行代码生成和调试
小程序开发全流程	✅ 掌握	理解从创建到预览的完整流程

6.2 原生小程序与其他开发模式对比

对比维度	原生小程序	Taro跨平台	混合开发
学习成本	需学习WXML/WXSS	使用React/Vue，学习成本低	需掌握双端知识
多端能力	仅微信小程序	一套代码多端运行	部分代码可复用
性能	最优	略低（编译转换开销）	中
包体积	小	较大（含运行时）	中
开发效率	AI辅助后显著提升	组件复用效率高	需分别适配
社区生态	微信官方生态成熟	第三方生态	依赖多种框架
调试便利性	微信开发者工具强大	需各平台分别调试	依赖不同工具链

6.3 AI工程铁律实践心得

- 提示词工程：**好的提示词是AI辅助开发成功的关键。需要详细描述技术栈、功能需求、数据结构和输出格式，避免模糊和歧义。
- 上下文管理：**在修复Bug时，提供完整的项目文件结构和相关代码上下文，AI能够更准确地定位问题。例如修复"小程序无法显示内容"问题时，将app.json、app.wxss和index.wxss一并提供给AI分析。
- 驾驭能力：**AI生成的代码需要人工审核和完善。例如搜索页的getAllNews()方法中数据重复问题，需要引导AI进行重构。
- 评估能力：**从正确性（功能是否实现）、可维护性（代码结构是否清晰）、性能（渲染是否流畅）、兼容性（不同设备表现）四个维度评估AI生成代码。
- 演进能力：**通过多轮迭代逐步完善代码，从基础列表到添加分类筛选、搜索、预览等功能，不断丰富应用。

6.4 改进建议

序号	改进项	状态	说明
1	数据层优化	已完成	将模拟数据提取为独立的 data/index.js 文件，通过模块化导出实现代码复用，遵循关注点分离原则
2	组件化开发	待实现	将通知卡片提取为独立组件，提高代码复用性
3	真实API对接	待实现	接入真实的校园通知API，替换模拟数据
4	性能优化	待实现	使用虚拟列表优化长列表渲染性能
5	用户系统	待实现	添加登录功能，实现个性化通知推送
6	消息推送	待实现	集成微信订阅消息，实现重要通知实时推送

6.5 代码重构总结

根据AI批阅建议，本次实验进行了以下代码重构：

重构前问题：通知数据在首页(pages/index/index.js)、详情页(pages/detail/detail.js)和搜索页(pages/search/search.js)中重复定义，导致代码冗余、维护困难。

重构方案：

1. 创建独立的数据模块 data/index.js ，统一管理通知数据和分类数据
2. 封装工具函数： getNewsById() 、 filterNewsByCategory() 、 searchNews()
3. 各页面通过 require('../../data/index.js') 引入共享数据模块

重构效果：

- 消除了数据重复定义，代码量减少约60行
- 提高了代码可维护性，数据修改只需在一处进行
- 遵循了关注点分离原则，数据层与业务逻辑层分离
- 便于后续扩展，如需接入真实API只需修改数据模块

实验报告生成时间：2026年4月28日

作者：汪明昊（2023210624）